Lab 3 – Architecture Justification

Topic: Virtual Technical Conference Platform

NAME-BHUVI BAGGA

SRN-PES2UG24CS117

CLASS -5B

We chose Microservices Architecture for the Virtual Technical Conference Platform System.

Architectural Choice

Microservices Architecture was selected over Layered and Client-Server architectures. The platform is decomposed into independently deployable services — Registration & Ticketing, Payment, Scheduling, Streaming, and a shared Database component — each communicating through well-defined interfaces rather than through a single shared codebase or a centralized server process.

Reason 1: Independent Scaling for Spiky, Uneven Load

A live technical conference produces extremely uneven demand: the Streaming Service and Chat/Q&A traffic can spike to thousands of concurrent connections the moment a popular session goes live, while Registration and Payment see comparatively steady, low-volume traffic. In a Layered or Client-Server architecture, the whole application (or the single server) would need to scale together, wasting resources on components that don't need it. With microservices, the Streaming Service alone can be scaled out with extra instances or CDN edge nodes during peak session times, while Registration and Payment stay small and cost-efficient.

Reason 2: Fault Isolation Keeps Sessions Running

If the Scheduling Service or a notification/feedback feature fails or is being redeployed, it must not interrupt an attendee who is already watching a live session. Microservices isolate failures to individual services — the Streaming Service keeps running independently of issues elsewhere in the system. A Layered architecture, by contrast, typically runs as a single deployable unit, so a bug or crash in one layer risks taking down the entire application, including active live sessions. A Client-Server design carries the same risk, concentrated in a single central server.

Security Advantage

Isolating the Payment Service as its own microservice limits the blast radius of a security incident. Payment card data and transaction logic can be placed behind stricter network segmentation, its own authentication scope, and a separate, more tightly audited data store — rather than sharing a database connection pool or process memory space with less sensitive services like Scheduling or the User Interface layer. If the Streaming Service is compromised, for example, an attacker gains no direct path to payment data, unlike in a monolithic Layered architecture where all business logic typically shares the same trust boundary.

Performance Benefit

The Streaming Service can be deployed and optimized independently for low-latency video delivery — for example, placed behind a content delivery network (CDN) or in regions closer to attendees — without being constrained by the deployment or performance profile of the Registration, Payment, or Scheduling services. This targeted optimization would be far harder in a Layered architecture, where the presentation, business, and data layers are typically deployed together and tuned as a single unit, making it difficult to give one function (video delivery) dramatically different performance treatment than the rest.

Component Diagram: Virtual Technical Conference Platform (Microservices Architecture)

